There are infinitely many primes

05 November 2026

Note : This is a demo post

Here's a theorem that everyone knows.

Theorem 1 *(Euclid)*: *There are infinitely many prime numbers.*

Proof Sketch: Suppose there were only finitely many primes, $p_1, \ldots, p_k$. The number $N = p_1 \cdots p_k + 1$ is greater than one, so it has a prime divisor q. None of the listed primes divides N: each leaves remainder one. Thus q is a prime missing from the list, a contradiction. ■

The Lean formalization

The formalization first constructs a prime $p \geq n$ for any bound n. It takes p to be the least prime factor of $n! + 1$. If $p \leq n$, then p divides both $n!$ and $n! + 1$, so it divides 1, contradicting its primality.

```
theorem prime_above (n : Nat) : ∃ p, n ≤ p ∧ Nat.Prime p := by
  let p := (n.factorial + 1).minFac
  have hne : n.factorial + 1 ≠ 1 := by
    have hpos := Nat.factorial_pos n
    lia
  have hp : Nat.Prime p := Nat.minFac_prime hne
  refine ⟨p, ?_, hp⟩
  by_contra h
  have hsmall : p ≤ n := by lia
  have hfac : p ∣ n.factorial := Nat.dvd_factorial hp.pos hsmall
  have hnext : p ∣ n.factorial + 1 := Nat.minFac_dvd _
  have hone : p ∣ 1 := (Nat.dvd_add_iff_right hfac).mpr hnext
  exact hp.not_dvd_one hone
```

Choosing the bound one greater than any proposed largest prime rules out a largest prime.

Finally, `Demo.infinitely_many_primes` uses this construction to rule out a finite set of primes: a finite set would have an upper bound, and `Demo.prime_above` supplies a prime beyond it.

```
theorem infinitely_many_primes : {p : Nat | Nat.Prime p}.Infinite := by
  by_contra h
  have hfinite : {p : Nat | Nat.Prime p}.Finite := Set.not_infinite.mp h
  obtain ⟨bound, hbound⟩ := hfinite.bddAbove
  obtain ⟨p, hlarge, hp⟩ := prime_above (bound + 1)
  have hle : p ≤ bound := hbound hp
  lia
```

Every double is even

02 November 2026

Note : This is a demo post

> **Theorem 2**: *For every natural number n, the number $n + n$ is even.*
>
> *Proof Sketch*: By definition, a number is even if it can be written as $k + k$ for some natural number k. Choose $k = n$ and the required equality is immediate. ■

A witness and an equality

In Lean, `Even (n + n)` asks us to supply that witness and its equality. The pair below chooses `n`; `rfl` proves the equality by reflexivity:

```
theorem double_is_even (n : Nat) : Even (n + n) := by
  exact ⟨n, rfl⟩
```

Here is the goal before we give Lean the witness:

```
n : ℕ
⊢ Even (n + n)
```

Swapping two summands

28 October 2026

Note : This is a demo post

> **Theorem 3** *(Commutativity of addition)*: *For any natural numbers m and n, we have $m + n = n + m$.*

This is the commutative law of addition: changing the order of the summands leaves their sum unchanged. For example, $3 + 5 = 5 + 3 = 8$.

The Lean formalization

> *Proof Sketch*: We prove the statement by induction on n. The zero case reduces to $m = m$. In the successor case, both sides reduce to the successor of the corresponding sum, so the induction hypothesis finishes the proof. ■

```
theorem addition_commutes (m n : Nat) : m + n = n + m := by
  induction n with
  | zero => simp
  | succ n ih => simp only [Nat.add_succ, Nat.succ_add, ih]
```

We can also ask Lean to calculate a concrete example:

```
#eval (3 + 5, 5 + 3)
```

```
(8, 8)
```